Aim of the Experiment: Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

```
import numpy as np
import matplotlib.pyplot as plt

def kernel(x, point, tau):
  """
  Calculates the kernel weight for a given point and input x.

  Args:
    x: The input data point.
    point: The point for which the kernel weight is calculated.
    tau: The bandwidth parameter.

  Returns:
    The kernel weight.
  """
  return np.exp(-((x - point)**2) / (2 * tau**2))


def locally_weighted_regression(x, X, Y, tau):
  """
  Performs locally weighted regression.

  Args:
    x: The input point for which to predict the output.
    X: The training data input features.
    Y: The training data output values.
    tau: The bandwidth parameter.

  Returns:
    The predicted output value for x.
  """
  weights = kernel(X[:, 0], x[0], tau) # Calculate weights using the first
                                       #column of X (the feature)
  W = np.diag(weights)
  beta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ Y
  return x @ beta
```

```
# Generate synthetic data
X = np.linspace(-3, 3, 50)
Y = np.sin(X) + np.random.randn(50) * 0.2
X = X.reshape(-1, 1)
X = np.concatenate((X, np.ones((50, 1))), axis=1)  # Add a bias term

# Choose a bandwidth parameter
tau = 0.5

# Perform locally weighted regression and plot the results
Y_pred = [locally_weighted_regression(x, X, Y, tau) for x in X] # Iterate
over each row of X

plt.scatter(X[:, 0], Y, label='Data')
plt.plot(X[:, 0], Y_pred, color='red',label='Locally Weighted Regression')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Locally Weighted Regression')
plt.legend()
plt.show()
```

Output: